Communications Systems Visualizer

ENGR 78 — Final Project Report

Howard Wang Kaw Moo Charlie Schuetz

Spring 2026

1 Introduction

1.1 Motivation

Undergraduate communications courses introduce a wide span of material—from sampling and quantization to line codes, pulse shaping, passband modulation, and information limits—often in separate homework sets or static figures. It is easy to memorize definitions without building intuition for how parameters (bandwidth, roll-off, noise, symbol rate) change time-domain waveforms, spectra, and eye openings in tandem. This project addresses that gap with a **single, locally hosted web application** that lets an instructor or student adjust controls and see linked plots update immediately.

1.2 Objectives

The team implemented a **Dash** application (Plotly front end, NumPy/SciPy back end) that:

- consolidates ENGR 78 topics into five topical tabs (PCM; line coding; ISI and eye diagrams; M-ary PAM and carrier modulation; capacity and simple coding);
- runs without specialized hardware in the default configuration;
- remains extensible via a clear module layout and an optional ADALM-PLUTO live path for demonstrations.

1.3 Scope and usage

From the repository root, install dependencies with `pip install -r requirements.txt`, then run `python main.py`. Open the URL shown in the terminal in a modern browser; the default address is `http://127.0.0.1:8050/`.

The UI uses `dcc.Tabs`. Each tab exposes sliders, dropdowns, or text inputs that drive `@app.callback` handlers in `web_app.py`. Plots are regenerated server-side and returned as Plotly figure objects.

For optional over-the-air or loopback experiments, run `python pluto_live_app.py` and open `http://127.0.0.1:8051/` (separate port from the class visualizer). Use **Simulation (no radio)** in that UI if hardware is unavailable, then **Start**. The second Dash application uses modules under `src/`, including `realtime_engine.py`, `modulation.py`, and `pluto_interface.py`. It is *not* required to use or grade the primary course visualizer.

Figures in Section 3 were exported to `docs/plot_guide_screenshots/` from the running application while preparing documentation; they are representative of the current UI theme and layout.

2 System description

2.1 Software stack

- **Dash** and **Plotly** provide layout, routing between tabs, and interactive graphics (pan, zoom, hover) without a separate desktop GUI framework for the main deliverable.
- **NumPy** implements discrete-time signals, FFT-based spectra, convolution for pulse shaping, and Monte Carlo draws for noise and random symbols in the eye-diagram tab.
- **SciPy** supports utilities such as windowed spectral estimates where applicable.

2.2 Repository layout

Table 1 summarizes the main entry points and supporting code. Legacy SDR-oriented modules remain under `src/` for the Pluto dashboard and for reference.

Path	Role
`main.py`	Launches the Dash development server (localhost; default port 8050).
`web_app.py`	Application layout, tab definitions, callbacks, and in-browser signal generation.
`pluto_live_app.py`	Optional live dashboard for ADALM-PLUTO streaming.
`src/realtime_engine.py`	Engine for streaming RX samples and metrics in Pluto mode.
`src/modulation.py`, `pluto_interface.py`, …	Modulation definitions and hardware/simulation bridge.
`requirements.txt`	Declares NumPy, SciPy, Dash, Plotly, and pyadi-iio.

Table 1: Primary modules for the communications visualizer.

2.3 Functional summary by tab

PCM. Users set message bandwidth, oversampling relative to Nyquist, and bits per sample; the interface reports implied sampling rate, quantization levels, and PCM bit rate alongside a conceptual waveform.

Line coding. A debounced binary string drives six line-code families (on-off and polar NRZ/RZ, AMI, Manchester). The tab shows the voltage waveform and a normalized PSD estimate from a windowed FFT.

ISI and eye. Raised-cosine roll-off β, optional neighbor-symbol coupling, additive noise, and PAM order control a random data eye diagram built from overlaid symbol-period segments.

M-ary and carrier. Symbol rate and PAM order illustrate rate–bandwidth tradeoffs. A dropdown selects ASK/OOK, BPSK, BFSK, QPSK, or 16-QAM; the tab shows passband time series, spectrum, and constellation or amplitude views where applicable, plus instantaneous and cumulative throughput.

Capacity and codes. For a chosen AWGN bandwidth and SNR, the Shannon capacity $C = B\log_2(1+\mathrm{SNR})$ is plotted with the operating point marked. User-specified source probabilities yield entropy $H(X)$. A repetition-code panel illustrates Hamming distance to codewords for a toy three-bit received word.

3 Results

Figure 1 shows the main layout with the carrier tab set to QPSK: the tab strip at the top, a row of summary statistics, and a grid of dark-theme Plotly panels. This layout is consistent across modulation choices; only the numerical readouts and traces change when the user switches schemes or moves sliders.

Figure 2 aggregates several representative panels—constellation, spectrum, eye diagram, throughput—into a single composite image. Such exports are convenient for report appendices or slide decks when full-page browser captures are not needed.

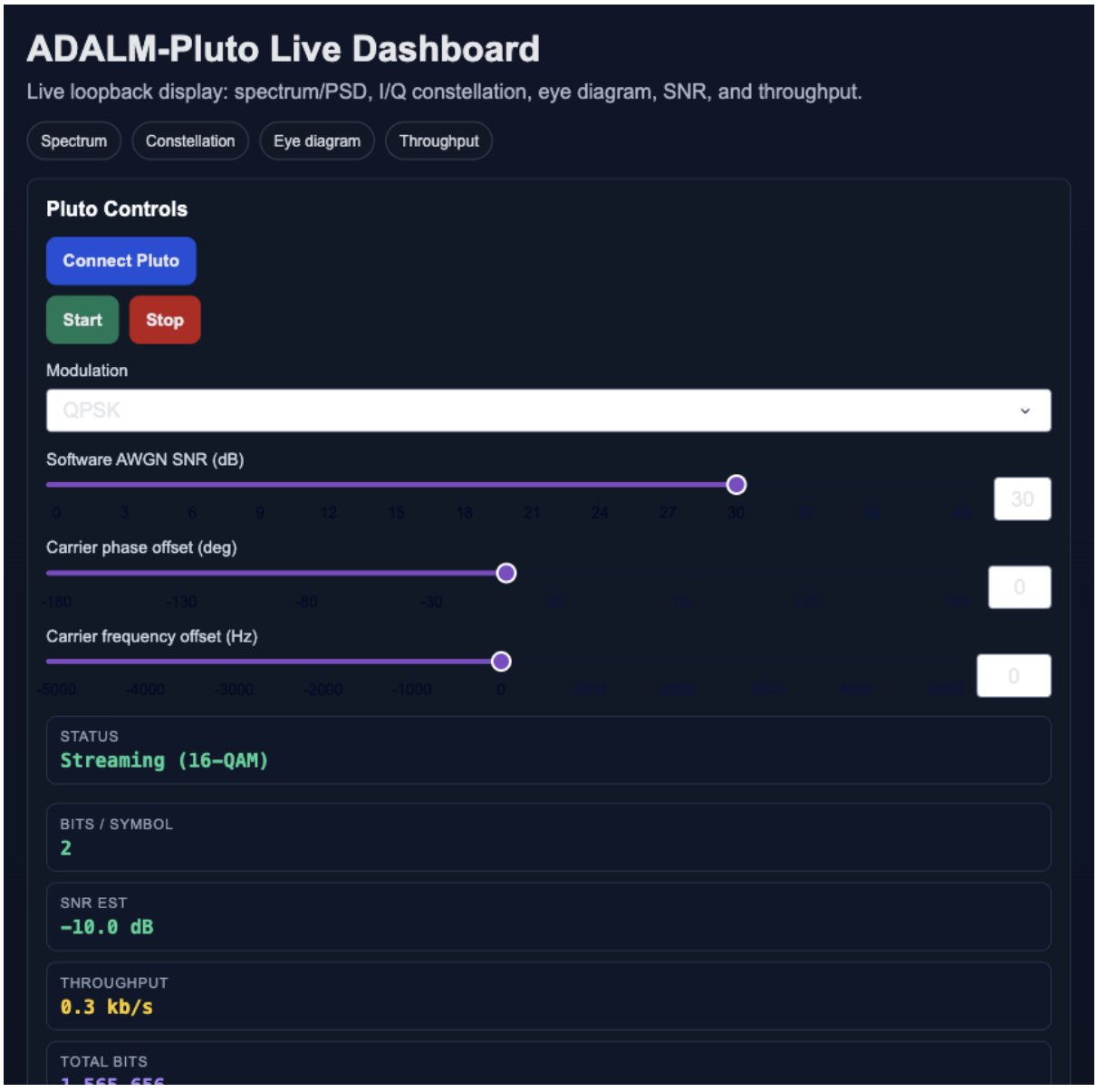


Figure 1: Dashboard with QPSK selected: tab navigation, statistics row, and Plotly panels.

Figures 3 and 4 compare full-height captures for binary and 16-ary signaling on the same interface. BPSK yields a one-dimensional decision space in the constellation view and a relatively open eye under default noise. 16-QAM shows the expected 4×4 grid in I/Q with proportionally higher bits per symbol; throughput readouts reflect the higher payload rate for the same symbol clock when the simulation is configured accordingly. Both figures illustrate that the tool scales visually from low-order to high-order constellations without changing the

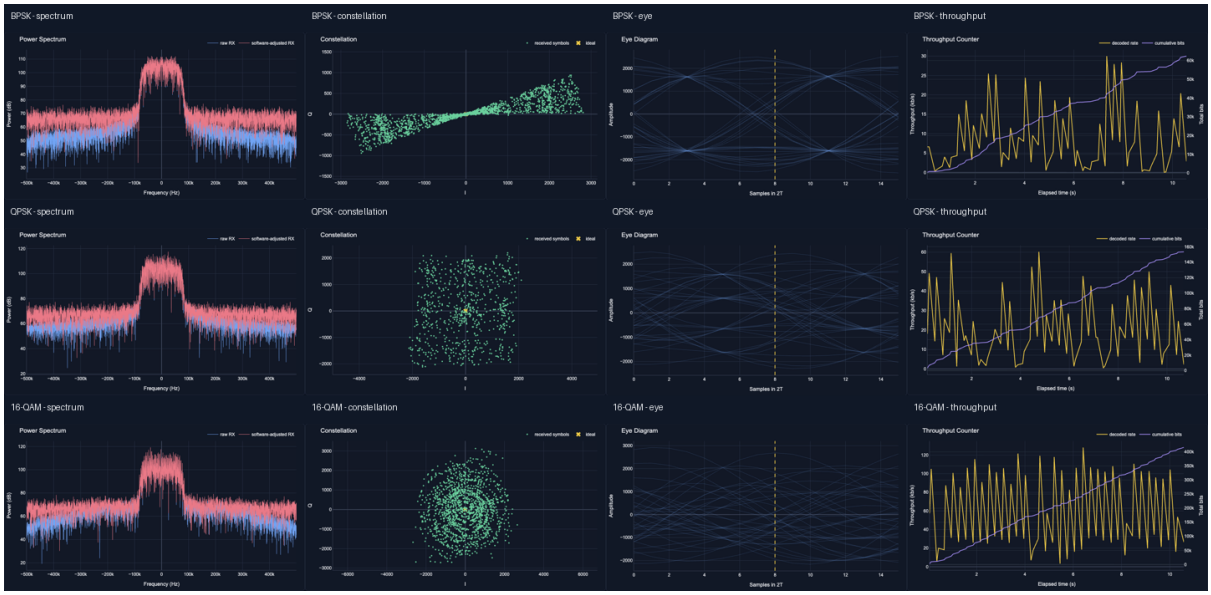


Figure 2: Composite export: constellation, spectrum, eye diagram, throughput, and related views.

overall navigation model.
Individual high-resolution crops in `docs/plot_guide_screenshots/` (e.g. `qpsk_constellation.png`, `qpsk_eye.png`, `16_qam_spectrum.png`) support selective inclusion in presentations.

4 Discussion and conclusion

The browser-first design met the practical requirement that every student reproduce the environment on a laptop: installation reduces to a Python virtual environment and `pip install -r requirements.txt`. Plotly's native interaction (zoom regions of the spectrum, inspect constellation clusters) supports exploratory use during office hours or lab sections.

Limitations are those of any server-side refresh model: many simultaneous callbacks and large FFTs can introduce latency on low-power machines; the Pluto path additionally depends on USB throughput and driver stability. Reasonable defaults in `web_app.py` (initial bit patterns, slider ranges) are important so the first screen is pedagogically meaningful without tuning.

Future work could include preset "lecture modes," CSV export of plotted traces, additional modulation or line-code options, or deeper error-control labs—all within the same Dash structure.

References

[1] Plotly Technologies Inc., *Dash documentation*. https://dash.plotly.com/

[2] Analog Devices, *ADALM-PLUTO user documentation*. https://wiki.analog.com/university/tools/pluto

[3] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed., McGraw-Hill, 2008.

ADALM-Pluto Live Dashboard

Live loopback display: spectrum/PSD, I/Q constellation, eye diagram, SNR, and throughput.

Spectrum | Constellation | Eye diagram | Throughput

Pluto Controls

Connect Pluto

Start | Stop

Modulation

BPSK

Software AWGN SNR (dB)

30

Carrier phase offset (deg)

0

Carrier frequency offset (Hz)

0

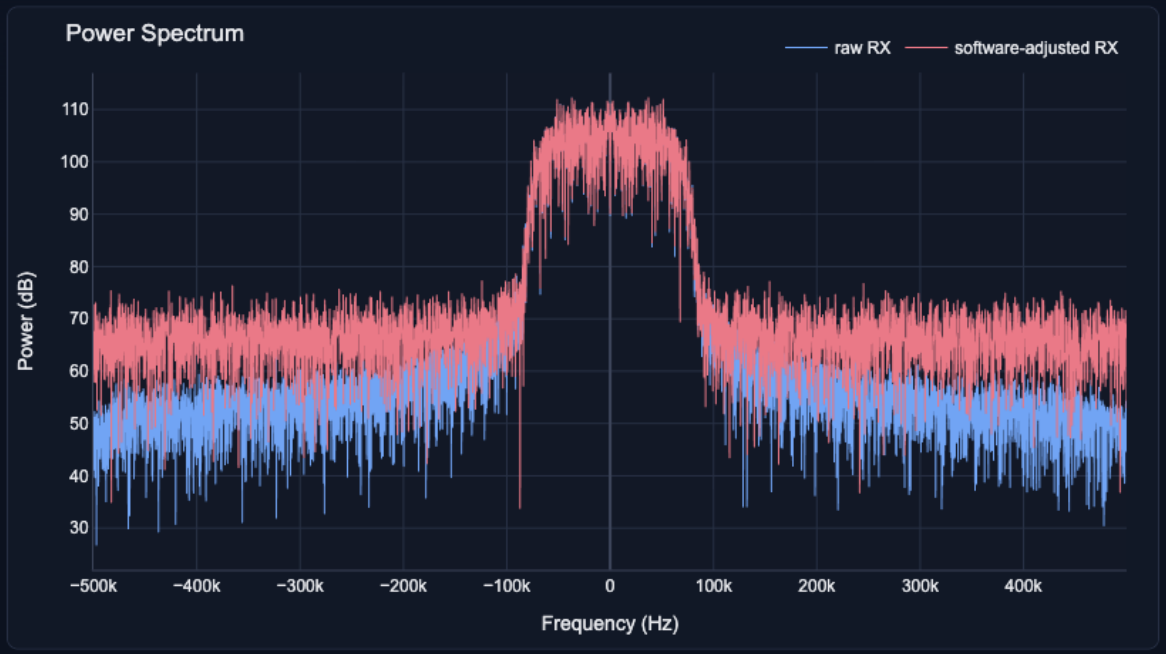
STATUS
Streaming (BPSK)

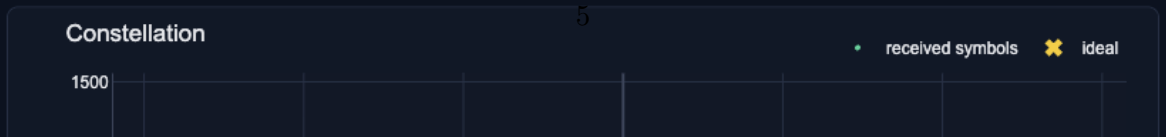
BITS / SYMBOL
1

SNR EST
-10.0 dB

THROUGHPUT
7.8 kb/s

TOTAL BITS
47,866





ADALM-Pluto Live Dashboard

Live loopback display: spectrum/PSD, I/Q constellation, eye diagram, SNR, and throughput.

Spectrum | Constellation | Eye diagram | Throughput

Pluto Controls

Connect Pluto

Start | Stop

Modulation

16-QAM

Software AWGN SNR (dB)

0 3 6 9 12 15 18 21 24 27 30 33 36 39 42

30

Carrier phase offset (deg)

-180 -130 -80 -30 20 70 120 170

0

Carrier frequency offset (Hz)

-5000 -4000 -3000 -2000 -1000 0 1000 2000 3000 4000 5000

0

STATUS

Streaming (16-QAM)

BITS / SYMBOL

4

SNR EST

-10.0 dB

THROUGHPUT

29.1 kb/s

TOTAL BITS

310,288